

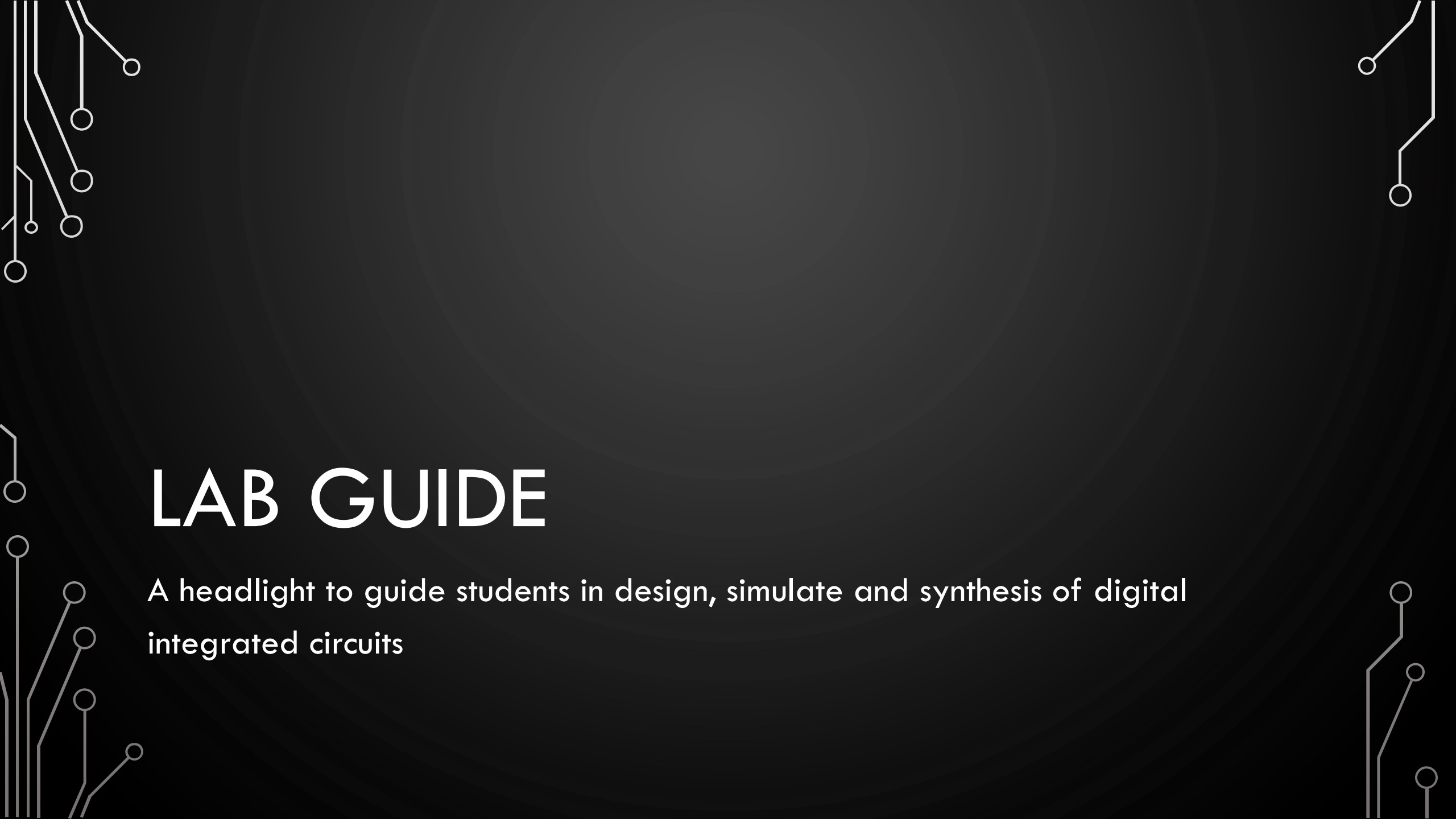


# DIGITAL SYSTEM DESIGN COURSE

FARSHAD BAHARVAND

ELECTRICAL ENGINEERING DEPARTMENT

SHARIF UNIVERSITY OF TECHNOLOGY

The background is a dark gray gradient with faint, concentric circular patterns. In the corners, there are white line art illustrations of circuit traces and nodes. The top-left and bottom-left corners have more complex, branching traces, while the top-right and bottom-right corners have simpler, more linear traces.

# LAB GUIDE

A headlight to guide students in design, simulate and synthesis of digital integrated circuits

The background is a dark gray with faint, concentric circular patterns. In the corners, there are white line art illustrations of circuit traces and nodes. Top-left: A cluster of lines with several circular nodes. Top-right: A few lines with circular nodes. Bottom-left: A cluster of lines with several circular nodes. Bottom-right: A few lines with circular nodes.

# SIMULATION

FUNCTIONAL AND POST-SYNTHESIS VERIFICATION METHODOLOGY

# Cadence “Xcelium” Simulator

- **What is it?** A **high-performance logic simulator** from Cadence, an industry-standard EDA tool used by companies like Apple, NVIDIA, and AMD to verify their chips.
- **Why do we need it?** Before building a physical chip (a very expensive process), we use Xcelium to **simulate our digital circuit's behavior** in software. It's like a flight simulator for digital logic.
- **What will it do for you?** Xcelium will take your Verilog/VHDL code and your testbench, and let you see exactly what's happening inside your design, signal-by-signal.

# Xcelium Key Features

- **Industry-Standard Language Support:** It understands the languages you're learning—**Verilog**, **SystemVerilog**, and **VHDL**—and even the advanced **UVM** methodology.
- **High Performance:** Its **multi-core simulation engine** is designed to be extremely fast, so you can quickly test your designs, even complex ones.
- **Powerful Waveform Debugging (SimVision):** This is your window into the circuit. Xcelium provides an integrated GUI tool called **SimVision** to view waveforms and debug your logic. You can see signals change over time, find glitches, and verify your design step-by-step.
- **Simplicity for Learning:** The tool provides a straightforward command-line interface (xrun) that follows a standard three-step flow: **compile**, **elaborate**, **simulate**.
- **Single-Command Simulation:** For simple designs, you can compile and run a simulation with a single command: **xrun <design\_files.v> -gui**

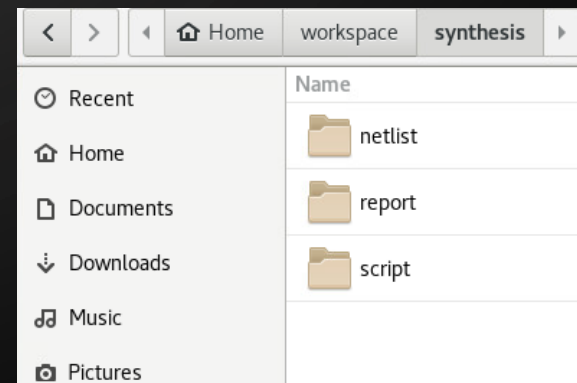
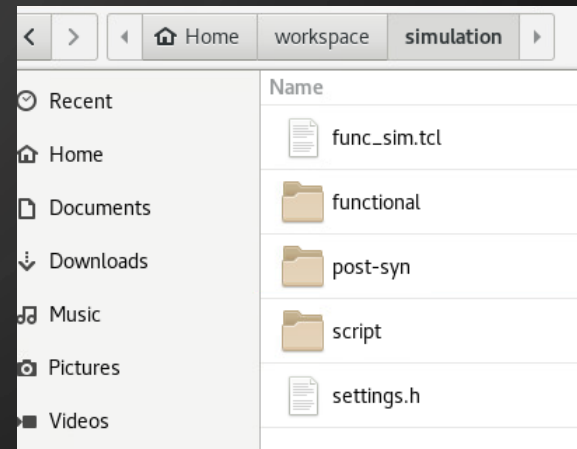
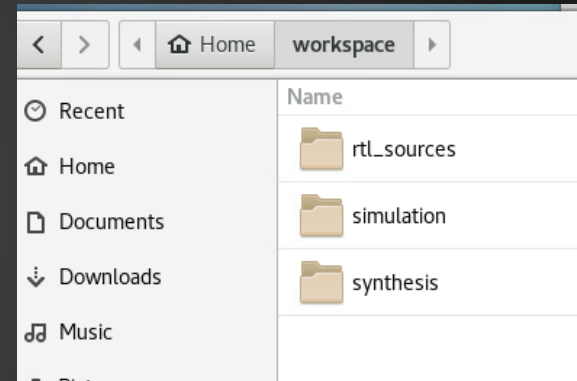
The background is a dark gray gradient with faint, concentric circular lines. In the four corners, there are white line-art illustrations of circuit traces and nodes, resembling a printed circuit board layout. These elements are positioned in the top-left, top-right, bottom-left, and bottom-right corners, framing the central text.

# STEPS TO SIMULATE YOUR DESIGN

# Simulation Steps: #1

## Preperation

- Create a **workspace** folder under Home folder
- Under workspace create these folders:
  - **rtl\_source**
    - Where are RTL including testbench resides
  - **simulation**
    - Where simulation will run, scripts and simulation logs exist
    - You enter this folder before starting simulator
  - **synthesis**
    - Where synthesis is done
    - The scripts, log files, reports and netlist are located here
- Simulation and synthesis scripts are different. Under each folder you'd better have a **script** folder



## Simulation Steps: #2

### Script Development

- Xrun is the command to run Cadence's Xcelium simulator
  - It compiles the sources and runs simvision gui to observe the hierarchy of modules as well as waveform window
  - Place the following command inside your script. The example script is called "**func\_sim.tcl**"

```
xrun -gui -ieee1364 -sv -disable_sem2009 -access +rwc -top <your top module name> \  
../rtl_sources/counter_display.v \  
../rtl_sources/testbench.v
```

- Under **simulation** folder run the script using linux **source** command

```
...simulation] $ source ./script/func_sim.tcl
```

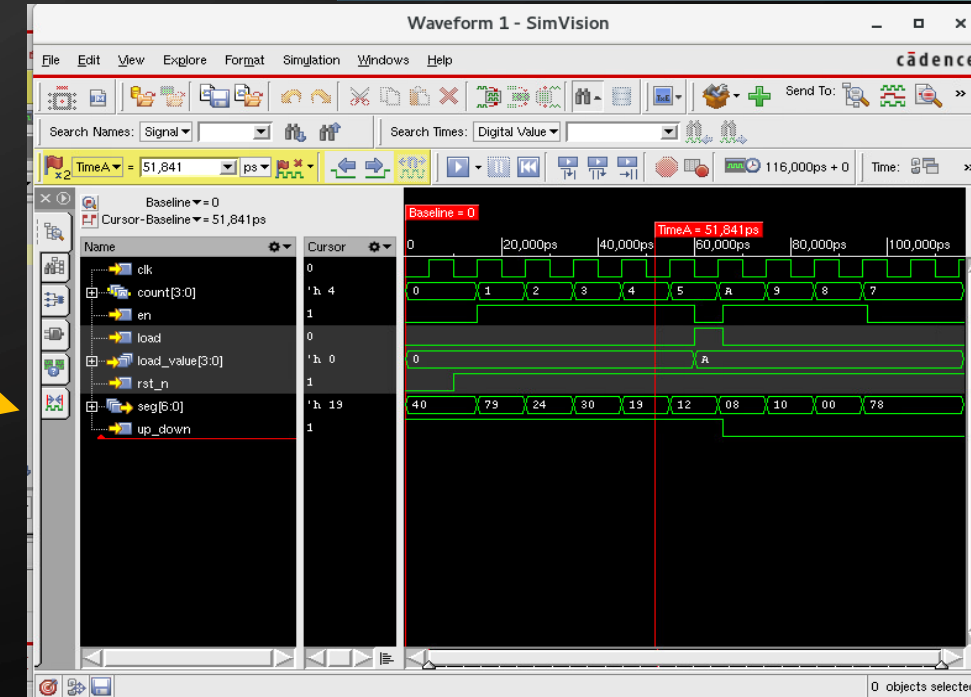
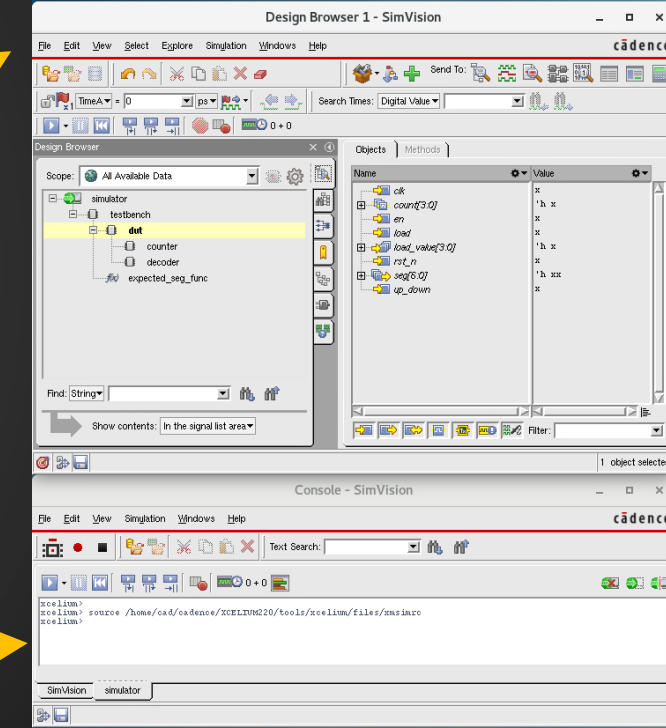
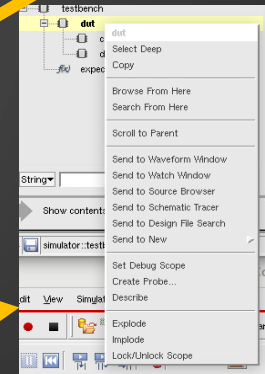
# Simulation Steps: #3

## SimVision

- SimVision “Design Browser” window shows the design hierarchy as well as signals in each level
- By right-click on each module you can add signals of the selected instance to the waveform window
- Under “Console” window you may run a simulation for 10 micro sec (10 us)

```
xcelium> run 10 us
```

- Waveform window shows the signals you have selected



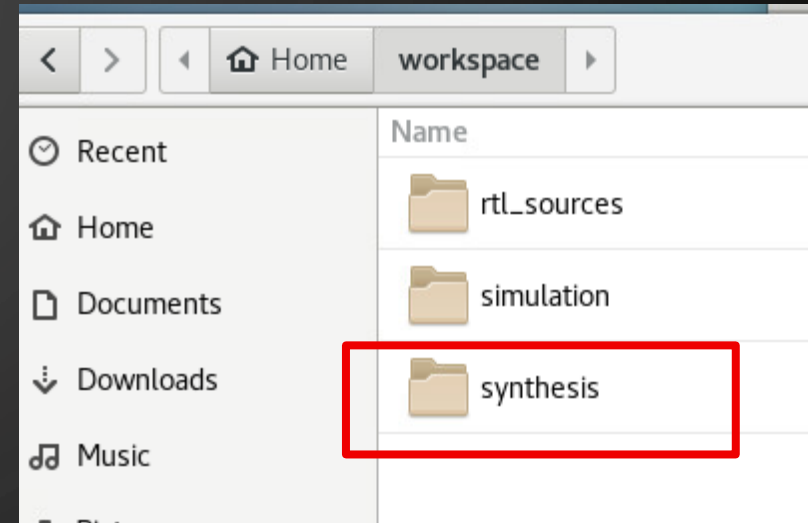
The slide features a dark gray background with a subtle pattern of concentric circles. In the four corners, there are white, stylized circuit-like lines and nodes, resembling a network or data structure.

# SYNTHESIS

MAPPING ONTO THE TECHNOLOGY

# Cadence Synthesis Solution

- *Genus Synthesis Solution*
- To run:
  - `$ cd workspace/synthesis`
  - `$ genus`
- Modify the synthesis scripts:
  - RTL file\_list
  - Clock and constraints definition
- Then run the main script in genus (no gui):
  - `source ./script/syn_scr.tcl`
  - This script calls:
    - `./script/lib_scr.tcl`
    - `./script/yourtopmodule_clock_const.tcl`



# Sections of synthesis script

```
#####
```

```
echo "-----"
echo "  Reading library"
echo "-----"
source ./script/lib_scr.tcl
```

Read Standard cell library

```
echo "-----"
echo "  Reading design RTL"
echo "-----"
```

Creating a variable that  
contains Verilog file list

```
# Read the Verilog design
set file_list {./rtl_sources/lfsr.v \
               ../rtl_sources/lfsr_top.v
              }
```

Setting how detail the  
synthesis log should be  
produced

```
##### synthesis attributes
set_db auto_ungroup none
set_db write_vlog_top_module_first true
```

```
#set_db write_vlog_preserve_net_name true
#set_db hdl_error_on_latch true
```

```
set_db information_level 1
#set_db use_tiehilo_for_const unique
set_db remove_assigns true
```

Reading the files and setting the  
language (e.g. SystemVerilog)

```
#set_db invs_assign_removal true
#set_db invs_assign_buffer auto
```

```
read_hdl -language sv $file_list
```

```
echo "-----"
echo "  Elaborating"
echo "-----"
```

Elaborating the design (resolving  
generic and parameters, expanding  
generate statements, resolving  
net/data types)

```
elaborate
```

# Sections of synthesis script

```
echo "-----"
echo " Setting top module and uniquifying"
echo "-----"

# Set the top-level module
set_top_module lfsr_top

#uniquify buf12_chip

echo "-----"
echo " Checking for unresolved references"
echo "-----"
check_design -unresolved

echo "-----"
echo " Define clocks and their constraints"
echo "-----"
source ./script/lfsr_clock_const.tcl

# MMC flow
#read_mmc ./scripts/mmc_scr.tcl
|
echo "-----"
echo " Initializing (loading) design"
echo "-----"
# initialize design - necessary for mmc flow
init_design
time_info init_design

echo "-----"
echo " Checking clocks definitions"
echo "-----"
report_clocks
```

Setting top module

Checking if there any  
unresolved module exists in  
design

Defining clocks (another script)

Checking if all clocks have  
been defined correctly

# Sections of synthesis script

```
echo "-----"
echo " Running synthesis"
echo "-----"

syn_generic
syn_map
syn_opt

echo "-----"
echo " Generating post-synthesis reports"
echo "-----"

#set_interactive_constraint_modes {constraint_mode_typ}

set_clock_latency -max 0.1 -clock clk -source clk -late

# Report the synthesis results
report_clocks
report_timing
report_area
report_power
report_gates

|
echo "-----"
echo " Wrting netlists and physical scripts"
echo "-----"

write_hdl > ./synout/lfsr_top_postsyn.v
write_sdf -version 3.0 -timescale ps -nonegcheck > ./synout/lfsr_top_postsyn.sdf

#write_sdc -view view_typ > ./lfsr_top.constraint_mode_typ.sdc

write_script > ./constraints.g

write_design -basename lfsr_top -tcf -innovus -hierarchical lfsr_top
```

Mapping to generic logic and

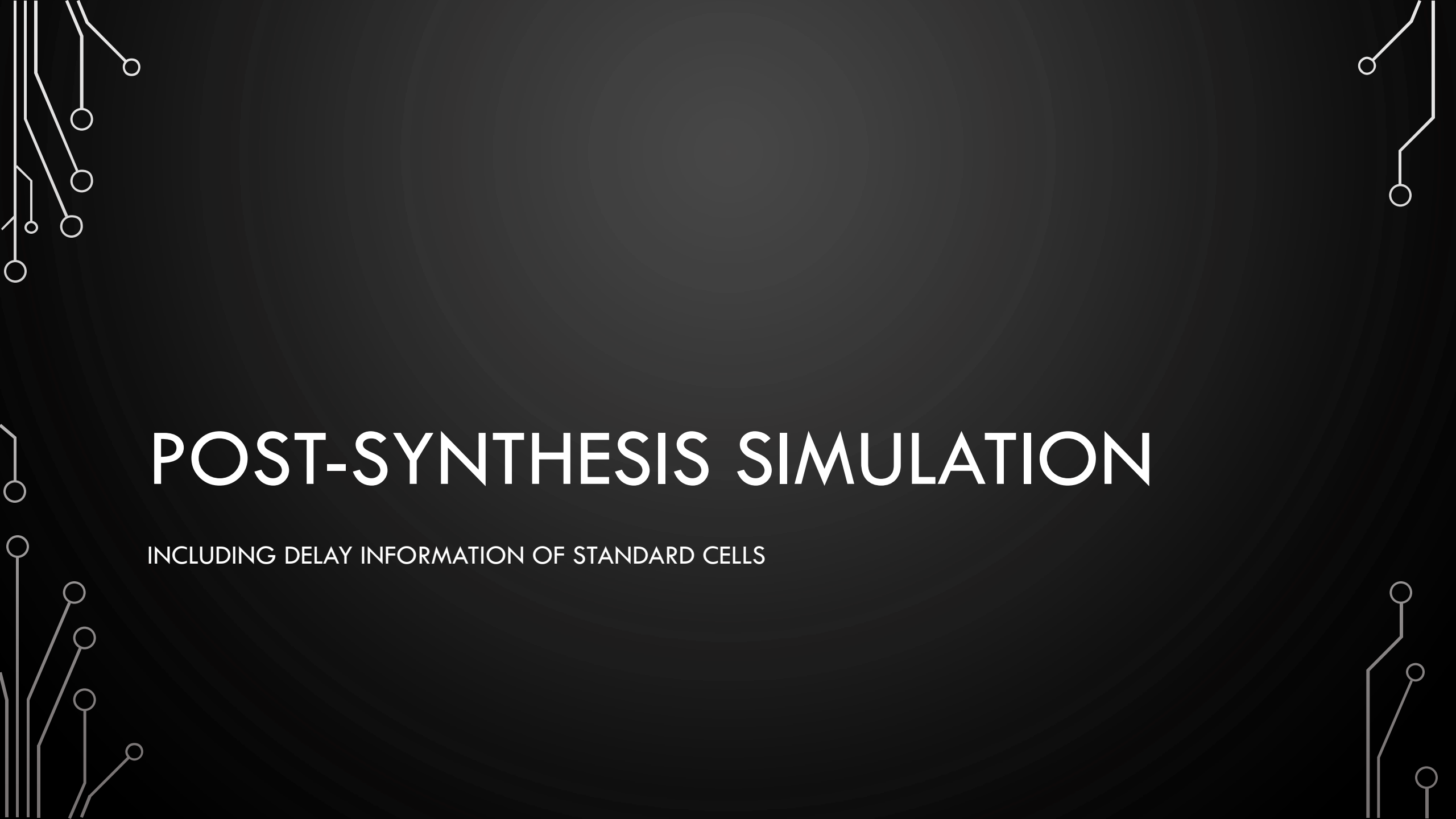
Mapping to standard cells

Logic optimization

Set latency on clock if necessary

Generating reports

Writing post-syn netlist and timing info

The background is a dark gray gradient with faint, concentric circles. In the corners, there are white line-art illustrations of circuit traces and nodes. Top-left: A cluster of lines with several circular nodes. Top-right: A few lines with circular nodes. Bottom-left: A more complex circuit-like structure with multiple nodes. Bottom-right: A few lines with circular nodes.

# POST-SYNTHESIS SIMULATION

INCLUDING DELAY INFORMATION OF STANDARD CELLS

# How to change the testbench for selective simulation

- Use `define to choose between functional and post-synthesis simulations
- If doing post-syn simulation read the standard cell Verilog library
  - If design has PADs then IO pad Verilog library need to be read as well
- Annotate SDF (Standard Delay Format) to the netlist
  - SDF was produced during synthesis
  - Should be done inside `initial` block
  - MAXIMUM says which item of triplet should be used as delay

```
`timescale 1ns/1ps
// Define if post synthesis simulation must be done or not. If not comment the following line
`define POSTSYN_SIM

//Testbench starts here

`ifndef POSTSYN_SIM
    `include "/home/cad/TECH.D/TSMC180/Verilog/tsmc18.v"
    `include "../synthesis/synout/lfsr_top_postsyn.v"
`else
    module lfsr #(
        parameter WIDTH = 8
        parameter [WIDTH-1:0] TAPS = 0
    ) (
        input wire clk,
        input wire rst_n,
        input wire en,
        input wire load,
        input wire [WIDTH-1:0] init_state,
        output reg [WIDTH-1:0] state,
        output wire [WIDTH-1:0] random_out
    );
    wire feedback;
    reg [WIDTH-1:0] tap_xor;
    // Generate loop to compute XOR of all tapped bits
    // Hidden requirement: use blocking assignment inside always @(*)
    integer i;
    always @(*) begin
        tap_xor = 0;
        for (i = 0; i < TAPS; i = i + 1)
            tap_xor = tap_xor ^ state[i];
        feedback = state[0] ^ tap_xor;
        state = {state[WIDTH-2:0], feedback};
    end
endmodule
```

```
reg [7:0] golden [0:19];
initial begin
    `ifndef POSTSYN_SIM
        $sdf_annotate ("../synthesis/synout/lfsr_top_postsyn.sdf", dut, , "MAXIMUM");
    `endif
    clk = 0;
    rst_n = 0;
    // Initialize golden sequence
    // Step0: after load = 10101010
    golden[0] = 8'b01011111;
    golden[1] = 8'b00101111;
```